NumPy and Pandas Fundamentals

AI Accelerator Program

Session 5 | 4:30 PM – 6:30

Instructor;
Mahi Aminu Aliyu
AI & SI Associate AHFID

Today's Focus

- Slides: concepts, mentai models, visuals
- Notebooks: full code + practice

Learning Objectives

By the end of this session, you should be able to:

- Explain what NumPy is (and why it's fast)
- Reason about arrays using `shape`, `dtype`, and `axis`
- Expiain what Pandas is (and why DataFrames are useful)
- Read basic EDA visuals from Nigerian-context datasets
- Choose when to use NumPy vs Pandas in a workflow

Big Picture: Where These Fit

```
Raw CSV / Excel
    I (Pandas: load + clean + join)
Clean table (DataFraMe)
    i (NumPy: fast numeric ops / matrix view)
Arrays / Matrices

Modeling (scikit-learn / TensorFlow)
```

Part 1 — NumPy

NumPy in One Sentence

NumPy gives you fast, typed, multi-dimensional arrays so you can do *math on whole datasets at once.*

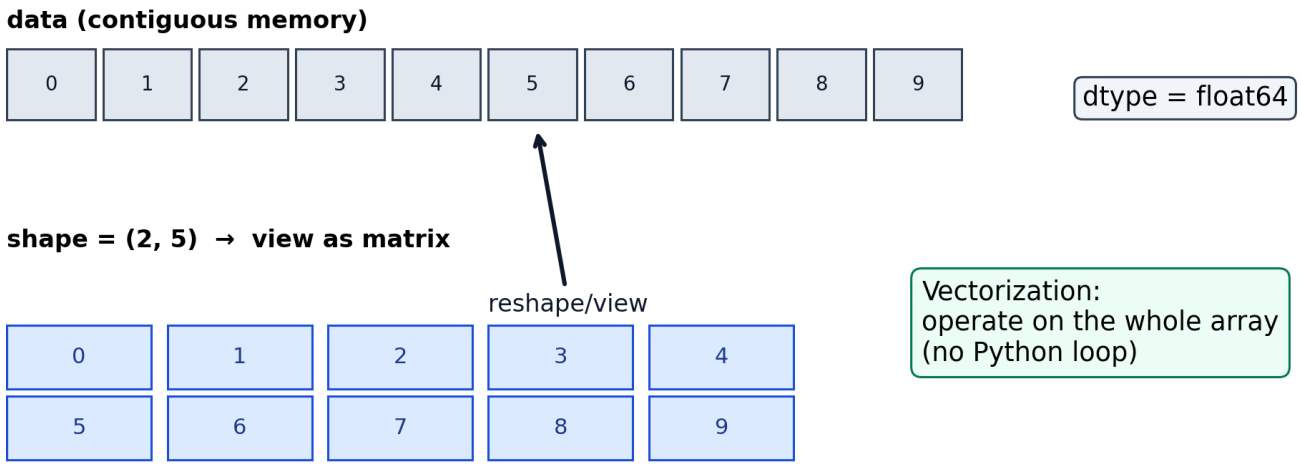
Why NumPy Exists (The “Why“)

- Python lists are filexible, but not optimized for numerical loops
- NumPy stores data in compact, contiguous memory
- NumPy operations run in compiled code (vectorized)

Practical result: fewer Python–level loops, more work done in optimized kernels.

The NumPy Mental Model

NumPy mental model: (data + dtype + shape)



Three Words That Explain Most Bugs

- dtype : what kind of numbers? (int, float, ...)
- shape : how many dimensions, and how big?
- axis *: which direction* are we aggregating over?

Illustrative Snippet (Tiny on Purpose)

```
import numpy as np
  x = np . array( [ [1,  2,  3]
                    [4,  5,  6]’]
                          )

x . shape            #  (2,  3)shape  (3,  )
x .mean(axis=0)  #  column  means
```

Broadcasting: The Core Trick

Broadcasting lets NumPy apply operations between arrays of different shapes.

Compatible when comparing from the end:

- dimensions are equal, or
- one dimension is 1

Broadcasting: Visual Intuition

```
A: shape (3, 4)
B: shape (4,)

B is treated like shape (1, 4)
and "stretched" to (3, 4) during the operation.
```

When NumPy is the Right Tool

- Numerical simulation / synthetic data generation
- Linear aigebra / matrix math (e.g., embeddings, PCA)
- Feature engineering with heavy numeric transforms
- Speed-critical preprocessing (vectorized operations)

Common NumPy Pitfalls (Conceptual)

- Silent dtype changes (ints vs floats)
- Views vs copies (modifying one changes the other)
- Axis confusion (row vs column operations)
- Shape mismatch during broadcasting

Part 2 — Pandas

Pandas in One Sentence

Pandas gives you a labeled table (DataFrame) that makes cieaning, joining, and summarizing real-world data practical.

Why Pandas is Useful (The ”Why“)

- Real data is messy: missing values, mixed types, timestamps
- Labels matter: columns and indexes make intent clear
- Built-in patterns for:
  - fiitering
  - group summaries
  - Joins/merges
  - time-series handiing

Mental Model: A DataFrame is a Table With Meaning

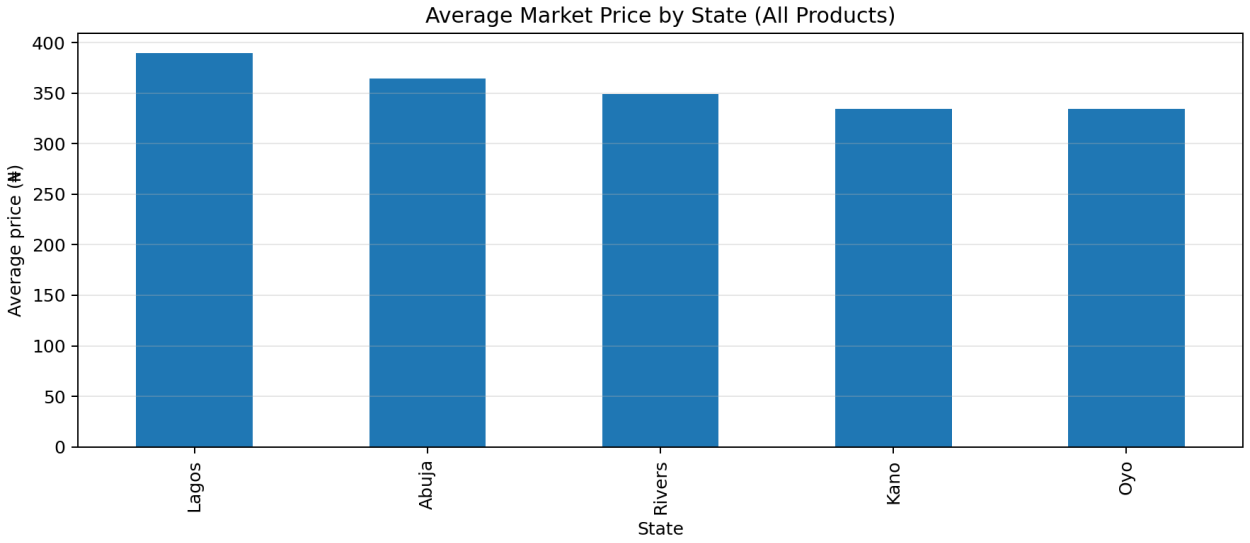
- Columns represent variabies/features
- Rows represent observations/records
- Index is a labeling system (sometimes meaningful, sometimes just row numbers)

Think: “spreadsheet with programmabie operations”.

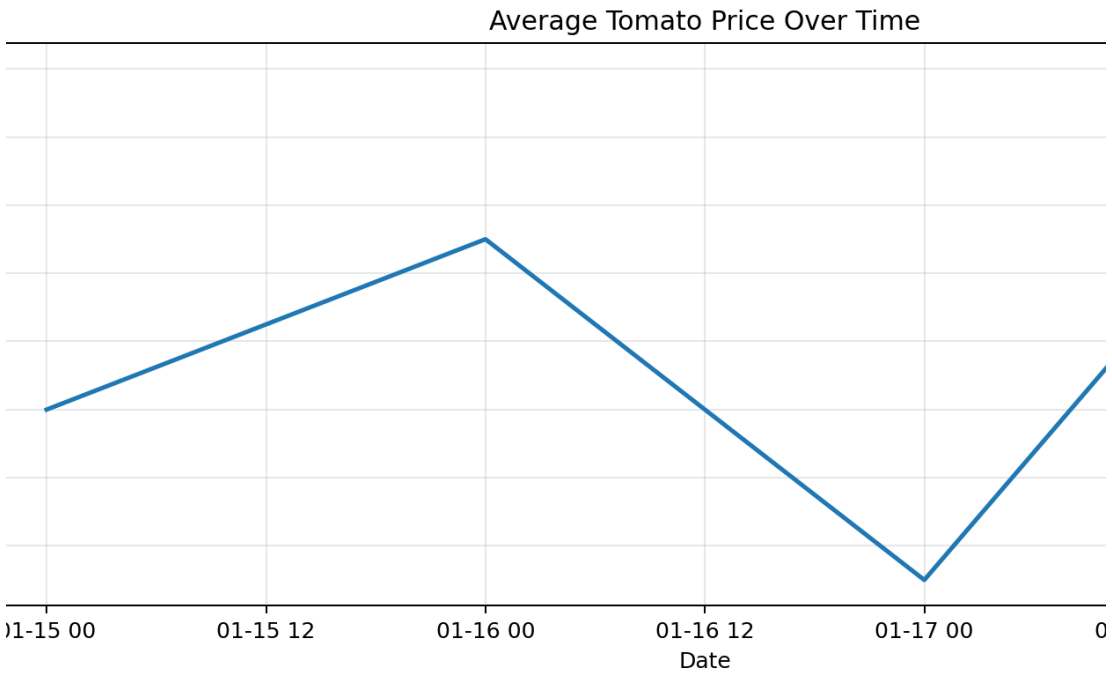
Visual: Average Market Price by State

Discussion prompts:

Which states look most expensive on average?



Visual: Price Trend Over Time



Discussion prompts:

- Is the trend stable or volatile?

Illustrative Snippet (Indexing Mental Model)

```
df.iloc[0:3]        # first 3 rows
df.loc[:, "price"] # column labeled "price"
```

GroupBy: Split w Apply w Combine

What it means conceptually:

1. split rows by a key (e.g., `state`)
2. compute something inside each group
3. combine results into a summary table

Tiny example:

```
avg = df.groupby("state")["price"].mean()
```

Merge/join: How You Answer “Real“ guestions

Many insights require combining tabies:

- market prices (by state/product/date)
- population (by state)
- income, infiation, rainfaii, iogistics costs, etc.

Common Pandas Pitfalls (Conceptual)

- Confusing `.loc` vs `.iloc`
- Chained assignment ("SettingWithCopyWarning")
- Treating dates as strings (harder time-series anaiysis)
- Assuming `mean()` is safe without checking missing values

NumPy vs Pandas: Quick Decision Guide

Use NumPy when:

- you already have clean numeric arrays
- you need fast math, matrix operations, vectorization

Use Pandas when:

- you are loading / cieaning / joining data
- you need labels (columns), group summaries, and fiiters

Most projects use both.

Practice (In Notebooks)

- Expiore the market dataset
- Compute summaries (by state, by product)
- Inspect missing values
- Create 1–2 visualizations to support a ciaim

Assignment (Overview)

- NumPy: synthetic numeric anaiysis (arrays, axis, broadcasting)
- Pandas: answer questions with fiitering + groupby + (optional) merge
- Insights: interpret results in a Nigerian context

Resources

- NumPy docs: https://numpy.org/doc
- Pandas docs: https://pandas.pydata.org/docs
- Nigerian data sources:
    - NBS: https://nigerianstat.gov.ng
    - HDX: https://data.humdata.org/group/nga

Good questions to ask:

- What is the "table" here, and what is the "matrix“?
- What does each axis represent?
- What would break my conclusion (missing data, outliers, coverage)?